

2026-09-28

Arity: A Standalone Language Server for R Package Development

R Consortium ISC proposal

Johan Larsson 

Department of Mathematical Sciences, University of Copenhagen

Executive Summary

R package development involves working across source files, dependencies, and documentation. I created [Arity](#) to help developers navigate and check this code from their editor without running R. Arity is an MIT-licensed language server written in Rust, with an integrated formatter and linter. It supports VS Code, Positron, Neovim, Zed, and other editors through the language server protocol (LSP).

This project will improve Arity's handling of shared documentation and function names, and provide function signatures and help for packages that are not installed locally, helping developers inspect unfamiliar code before installing its dependencies. I will publish the metadata and extraction pipeline for reuse by other R tools, and test the improvements in complete editing workflows. I ask for USD 10,000 to fund 125 hours of labor, targeting completion by April 30, 2027.

Signatories

Project team

I am Arity's maintainer and a postdoctoral researcher in statistics and machine learning at the University of Copenhagen's Department of Mathematical Sciences. I also develop and maintain several R packages, including [eulerr](#), [SLOPE](#), and [qualpalr](#).

Contributors

I prepared this proposal, incorporating feedback from the people acknowledged under [Consulted](#).

Consulted

The proposal draft was published in a [public GitHub repository](#) and I invited user feedback through social media and the issue tracker for Arity. I'm grateful for feedback from Jan Hartman, Josiah Parry and Tatsuya Shima (maintainer of [arf](#), [r-documentation-rs](#), and [mini-roxygen](#)).

The Problem

Changing a function in an R package can affect its documentation and calls across several files. Developers need reliable checks and navigation to make these changes safely, but Arity still has gaps: it can misinterpret shared roxygen documentation, miss undocumented parameters, and handle references and call hierarchy inconsistently. Renaming can also produce invalid R syntax, for example when a function is used as an infix operator.

Exploring unfamiliar package code is also harder when its dependencies are not installed locally. Arity provides signatures and help for installed packages, but only exported names for others. Richer metadata would let developers inspect function signatures and documentation before installing dependencies and enable linting of deprecated and defunct functions.

Arity is one of three language servers for R, alongside [languageserver](#) and [Ark](#). The R package `languageserver` runs in an R session, whereas Ark embeds its language server in an R kernel and powers R support in Positron. Arity is a standalone language server that works without an R session or Jupyter kernel.¹

The Proposal

Overview

This project will help R developers understand and change package code from their editor without running R. I will improve Arity's handling of shared documentation, make references and call hierarchy consistent, and ensure that renaming either produces valid R syntax or is refused. Developers will also gain function signatures and help for dependencies they have not installed, making unfamiliar projects easier to explore.

The work will produce a public metadata index for at least 100 CRAN packages, together with a documented format and extraction pipeline that other R tools can reuse. From December 2026 through April 2027, I will implement these improvements and test complete editing workflows on ten packages. Published examples and before-and-after results will show what developers can rely on and where limitations remain.

Detail

The full project covers three areas:

Language-server reliability I will improve analysis of shared and inherited roxygen documentation, navigation for quoted names, and renaming, including infix calls.

¹Adapting Ark to run independently of R would have required extracting the server and replacing its dependencies on the R runtime, which is what motivated the development of Arity.

Package knowledge Arity already extracts metadata without running R and bundles exported names for over 500 packages, refreshed weekly. I will extend its publishing pipeline and remote client to provide richer metadata for at least 100 CRAN packages, adding symbol kinds, signatures, help, and explicit deprecation information.

These indexes will support completion, hover, signature help, and deprecation diagnostics without local installation. Downloads will remain an explicit user choice and use Arity’s existing disk cache. Indexes will record package versions and extraction methods.

For the pilot, I will extract metadata from unpacked [R-universe binary packages](#), falling back on packages installed in continuous integration (CI). I will check metadata against the same package versions in R and use roxygen2 to check documentation behavior. Source packages and extracted data will retain version and license information.

To make the extraction code reusable by other R tools, I am coordinating with the maintainer of [r-documentation-rs](#) to bring our code for reading R binary data and Rd documentation into that library.

Initial extraction checks in September 2026 found the required fields for every exported function in glue, withr, R6, and cli. For magrittr, coverage was 16 of 42 functions, largely because primitive aliases lack formal arguments. At this coverage, magrittr would not meet the project’s 80% threshold for inclusion in the 100-package target. The checks also found S3 methods incorrectly listed as exports.

Validation and documentation I will test ten package workflows covering cross-file references, imports, shared documentation, and nonstandard evaluation, and publish results and guides.

Minimum Viable Product

The smallest useful release will correctly handle shared and inherited roxygen documentation and detect undocumented parameters. These fixes give package authors more reliable documentation checks and can be released independently of the navigation and metadata work.

Risks and Scope

Static analysis cannot resolve every name. Arity will refuse uncertain renames and withhold deprecation diagnostics when function resolution or the indexed deprecation status is uncertain. Indexes will initially cover current releases. When dependency versions cannot be matched, deprecation findings will identify the indexed release and remain advisory.

I will document unsupported cases, missing fields, and failed extractions. Older metadata will retain the package version it describes. Historical releases, index selection from lockfiles, and argument-specific deprecations are outside scope. Extracting function signatures by querying a running R session is, however, outside scope.

If full consolidation exceeds the available hours, I will retain the necessary parts of Arity’s extractor and defer the remaining consolidation work.

Project Plan

Start-Up Phase

I will begin working on the first milestone (M1) in December 2026, after the grant agreement. I will pin versions of ten evaluation packages, define their editing workflows, and record baseline diagnostics and response times.

I will track tasks and milestones in [Arity’s public GitHub repository](#). Monthly progress notes will record completed work, hours spent, risks, and any changes to the schedule. I will publish these notes in the repository and share them with the ISC, linking to tests, results, and releases as milestones are completed.

During M1, I will fix the S3 export errors and test extraction of deprecation declarations. I will define the supported cases before the metadata publication milestone (M4).

Before M4, five pilot packages must pass the metadata quality criteria listed under “Package knowledge” in [Definition of Done](#). A prototype using Arity’s existing client must provide signatures and help without local installation for packages both inside and outside the existing bundle. If the pilot fails to meet these requirements within budget, I will agree on a revised scope for the metadata milestones with the ISC.

Technical Delivery

I plan to spend about 25 hours per month on the project, completing the 125 hours by April 30, 2027. See [Table 1](#) for the milestones, target dates, deliverables, estimated hours, and costs.

Table 1: Milestones for the project, with target dates, deliverables, estimated hours, and costs.

Milestone	Target date	Deliverables	Hours	Cost (USD)
M1: Baseline and pilot	Jan 7, 2027	Baseline, metadata prototype, consolidation plan, quality checks, regression cases	30	2,400
M2: Documentation analysis	Jan 19, 2027	Correct handling of shared and inherited roxygen documentation and detection of undocumented parameters	10	800
M3: Navigation and safe renaming	Feb 6, 2027	Consistent references and call hierarchy for quoted names, safe renaming including infix calls	15	1,200
M4: Reusable package metadata	Feb 28, 2027	Indexes for at least 100 CRAN packages meeting the quality criteria, documented schema, extraction pipeline, weekly updates	20	1,600
M5: Package metadata in Arity	Mar 25, 2027	Completion, hover, signature help, deprecation diagnostics, bulk prefetch, shared offline cache for editor and CLI	20	1,600
M6: Validation and release	Apr 30, 2027	Workflow validation, annotated comparison, guides, release, report, maintenance instructions	30	2,400
Total			125	10,000

Other Aspects

Code, tests, and the metadata schema will use the MIT license. I will publish the schema, extraction and publishing pipeline, documentation, and results in Arity’s GitHub repository, along with

instructions for rebuilding or mirroring the dataset. Contributors can submit bug reports, suggestions, and patches through GitHub issues and pull requests.

I will offer the R Consortium blog an announcement in December 2026, a progress update in February 2027, and a results post in April 2027, with examples and remaining limitations. I will also publish updates on [my blog](#), whose R feed is included in [R Weekly's feed list](#). I will share these posts through social media and offer an update at an ISC meeting if requested. A useR! presentation would be a possible follow-up after the project.

Budget & Funding Plan

I intend to receive the grant personally. The requested USD 10,000 will fund 125 hours of my time at USD 80 per hour. The budget assumes no other funding or contributors. With partial funding, the ISC could select implementation milestones according to their value to the R community and I would adjust the scope and budgets of M1 and M6 to cover the selected work.

Success

Definition of Done

The project will be complete when I have published a tagged Arity release and met the following criteria:

Language-server reliability Tests show that Arity resolves shared documentation, reports undocumented parameters, and handles references and call hierarchy consistently for quoted function names. For supported quoted-name and infix cases, renaming updates definitions and statically resolved references, leaves unrelated bindings unchanged, and produces valid R syntax. Tests also verify that Arity refuses uncertain rewrites.

Package knowledge Indexes for at least 100 CRAN packages are published, with a documented schema and weekly updates. Export lists must match R's results for the same versions. I will compare a sample of extracted signatures and help text against the same package versions in R and withhold fields found to be incorrect or uncertain. After withholding incorrect or uncertain fields, each qualifying index must provide arguments, defaults, and help for at least 80% of exported functions. Missing fields will be reported.

Tests show completion details, hover information, and signature help without local installation, including packages outside the bundle. They also verify diagnostics for deprecated and defunct functions, covering supported declarations, local shadowing, and advisory behavior when dependency versions cannot be matched. The same tests pass offline after prefetch. For the supported deprecation cases, editor and command-line checks report the same diagnostics when given identical source, configuration, and cached metadata.

Validation and documentation I have published workflows and before-and-after results for ten packages, including checks in VS Code and Neovim, along with editor and CI guides and maintenance instructions. The release passes the existing workspace and corpus checks with no unexplained regressions.

Measuring Success

The report will compare diagnostics and response times before and after the project using the ten package workflows. I will classify sampled diagnostic findings as defects, false positives, or unresolved cases, and report remaining limitations and any new regressions. For metadata, I will report coverage, accuracy, how current the data is, and extraction failures, with deprecation coverage reported separately. The report will document the versions, configuration, and measurement setup so others can reproduce the comparison.

Future Work

Future work could cover more packages, select exact versions from a project's lockfile, support the CRAN Archive with package metadata from [ALLPACKAGES](#), and use runtime tracing.